

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»**

(Университет ИТМО)

Факультет Прикладной информатики

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

**ОТЧЕТ ПО Лабораторной работе 5 Проектирование и реализация баз
данных**

Тема: «Процедуры, функции, триггеры в PostgreSQL»

Обучающийся: Ипатова Ульяна Юрьевна К3239

Санкт-Петербург 2026

Цель работы: овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Ход работы:

Перед основной работой создаю таблицу логов:

```
bus_station=# CREATE TABLE IF NOT EXISTS bus_station_sch.logs(text text, added timestamp without time zone);  
CREATE TABLE  
bus_station=# |
```

Нужно создать 3 процедуры:

- Продажа билета.
- Возврат билета.
- Добавление нового рейса.

Процедура продажи билета:

- То есть она оформляет новый билет: сама создает новый номер билета, рассчитывает цену по тарифу и добавляет запись в таблицу `bilet` со статусом `PAID`

Смотрит:

Дату покупки билета.

Способ покупки билета.

Номер рейса, на который покупается билет.

Номер пассажира, который покупает билет.

Остановка, связанная с билетом.

Есть ли у пассажира ручная кладь.

Номер тарифа.

В начале процедура формирует новый номер билета как максимальный существующий `ticket_ID` плюс 1. Если билетов в таблице еще нет, номер первого билета принимается равным 1. Далее процедура ищет активный тариф, который соответствует переданному `tariff_id` и действует на дату покупки билета. Из тарифа берутся базовая цена билета, стоимость ручной клади и процент скидки для билета без места. После этого процедура рассчитывает итоговую стоимость билета. Если билет оформляется без места, то из базовой цены вычитается скидка. Если у пассажира есть ручная кладь, то к цене билета добавляется стоимость ручной клади. Если подходящий тариф найден, процедура добавляет новую запись в таблицу `bilet` со статусом `PAID`. Таким образом, процедура автоматизирует процесс продажи билета и расчет его стоимости.

CREATE OR REPLACE PROCEDURE

```
bus_station_sch.pr_sale_ticket(p_place_number integer,p_purchase_date
date,p_purchase_method character(1),p_flight_id integer,p_id_passenger
integer,p_stop_id integer,p_tariff_id integer,p_has_hand_luggage boolean)
LANGUAGE plpgsql AS $$ DECLARE new_ticket_id integer; new_ticket_price
numeric(8,2); base_price numeric(8,2); luggage_price numeric(8,2); discount
numeric(5,2); BEGIN SELECT max("ticket_ID") + 1 INTO new_ticket_id FROM
bus_station_sch.bilet; IF new_ticket_id IS NULL THEN new_ticket_id := 1; END
IF; SELECT
"base_ticket_price","hand_luggage_price","no_seat_discount_percent" INTO
base_price,luggage_price,discount FROM bus_station_sch.tarif_bileta WHERE
"tariff_id" = p_tariff_id AND "is_active" = true AND p_purchase_date >=
"valid_from" AND ("valid_to" IS NULL OR p_purchase_date <= "valid_to"); IF
base_price IS NOT NULL THEN IF luggage_price IS NULL THEN
luggage_price := 0; END IF; IF discount IS NULL THEN discount := 0; END IF;
new_ticket_price := base_price; IF p_place_number IS NULL THEN
```

```
new_ticket_price := new_ticket_price - new_ticket_price * discount / 100; END
IF; IF p_has_hand_luggage = true THEN new_ticket_price := new_ticket_price +
luggage_price; END IF; INSERT INTO
bus_station_sch.bilet("ticket_ID","place_number","purchase_date","ticket_price","
purchase_method","flight_ID","ID_passenger","stop_ID","ticket_status","tariff_id
","has_hand_luggage")
VALUES(new_ticket_id,p_place_number,p_purchase_date,new_ticket_price,p_pu
rchase_method,p_flight_id,p_id_passenger,p_stop_id,'PAID',p_tariff_id,p_has_han
d_luggage); END IF; END; $$;
```

Процедура возврата билета:

Процедура будет предназначена для оформления возврата билета.

На вход процедура получает номер билета p_ticket_id. После вызова процедуры выполняется поиск билета с указанным номером в таблице билет. Если такой билет существует и его текущий статус равен PAID, то есть билет оплачен, процедура изменяет его статус на RETURNED.

Таким образом, возврат билета реализуется не удалением записи из таблицы, а изменением статуса билета. Это позволяет сохранить информацию о проданном билете в базе данных и учитывать историю операций.

Если билет с указанным номером не найден или его статус уже отличается от PAID, процедура не изменяет данные.

CREATE OR REPLACE PROCEDURE

```
bus_station_sch.pr_return_ticket(p_ticket_id integer) LANGUAGE plpgsql AS $$  
BEGIN UPDATE bus_station_sch.bilet SET "ticket_status" = 'RETURNED'  
WHERE "ticket_ID" = p_ticket_id AND "ticket_status" = 'PAID'; END; $$;
```

Процедура добавления нового рейса:

Процедура предназначена для добавления нового рейса в таблицу reis.

На вход процедура получает дату отправления, время отправления, тип рейса, номер автобуса и номер пункта назначения.

Сначала процедура выполняет проверки. Она проверяет, существует ли автобус с указанным номером в таблице avtobus. Затем проверяется, существует ли указанный пункт назначения в таблице punkt_naznacheniya. После этого процедура проверяет, не назначен ли уже этот же автобус на другой рейс в ту же дату и то же время. Также проверяется тип рейса: он должен принимать значение R или D, где R - регулярный рейс, а D - дополнительный рейс. Если все успешно, процедура формирует новый номер рейса. Для этого она находит максимальный существующий flight_ID в таблице reis и прибавляет к нему 1. Если таблица рейсов пустая, то новому рейсу присваивается номер 1. После этого процедура добавляет новую запись в таблицу reis с полученными данными.

Если кратко, то процедура автоматизирует добавление рейса и не позволяет добавить рейс с несуществующим автобусом, несуществующим пунктом назначения, неправильным типом рейса или с автобусом, который уже занят в указанную дату и время.

CREATE OR REPLACE PROCEDURE

```
bus_station_sch.pr_add_flight(p_departure_date date,p_departure_time
time,p_flight_type character(1),p_bus_id character(10),p_destination_id integer)
LANGUAGE plpgsql AS $$ DECLARE new_flight_id integer; c1 integer; c2
integer; c3 integer; BEGIN SELECT count(*) INTO c1 FROM
bus_station_sch.avtobus WHERE "bus_ID" = p_bus_id; SELECT count(*) INTO
c2 FROM bus_station_sch.punkt_naznacheniya WHERE "destination_ID" =
p_destination_id; SELECT count(*) INTO c3 FROM bus_station_sch.reis
WHERE "bus_ID" = p_bus_id AND "departure_date" = p_departure_date AND
"departure_time" = p_departure_time; IF c1 = 1 AND c2 = 1 AND c3 = 0 AND
p_flight_type IN ('R','D') THEN SELECT max("flight_ID") + 1 INTO
new_flight_id FROM bus_station_sch.reis; IF new_flight_id IS NULL THEN
new_flight_id := 1; END IF; INSERT INTO
bus_station_sch.reis("flight_ID","departure_date","departure_time","flight_type","
bus_ID","destination_ID")
VALUES(new_flight_id,p_departure_date,p_departure_time,p_flight_type,p_bus_
id,p_destination_id); END IF; END; $$;
```

Триггеры:

Проверка периода продажи билета:

билет нельзя купить раньше, чем за 10 дней до рейса и нельзя купить после даты рейса

```
CREATE OR REPLACE FUNCTION bus_station_sch.fn_bilet_sale_period()
RETURNS trigger AS $$ DECLARE flight_date date; BEGIN IF
NEW."ticket_status" = 'PAID' THEN SELECT "departure_date" INTO flight_date
FROM bus_station_sch.reis WHERE "flight_ID" = NEW."flight_ID"; IF
flight_date IS NULL THEN RETURN NULL; END IF; IF NEW."purchase_date"
< flight_date - 10 THEN RETURN NULL; END IF; IF NEW."purchase_date" >
flight_date THEN RETURN NULL; END IF; END IF; RETURN NEW; END; $$
LANGUAGE plpgsql;
```

```
CREATE TRIGGER tr_bilet_sale_period BEFORE INSERT OR UPDATE ON
bus_station_sch.bilet FOR EACH ROW EXECUTE PROCEDURE
bus_station_sch.fn_bilet_sale_period();
```

ПРОВЕКА:

Для первого рейса departure_date = 2026-04-10, значит билеты на него можно купить только с 2026-03-31 по 2026-04-10 включительно.

flight_ID	departure_date	first_sale_date	purchase_date
1	2026-04-10	2026-03-31	2026-04-01

(1 строка)

Для проверки я беру дату покупки: 2026-03-30

Она раньше разрешенного периода, значит триггер должен отменить добавление билета.

Для этого вызову процедуру:

```
CALL bus_station_sch.pr_sale_ticket(8,'2026-03-30', 'K',1,5,3,1,false);
```

```
bus_station=# CALL bus_station_sch.pr_sale_ticket(8,'2026-03-30', 'K',1,5,3,1,false);
CALL
bus_station=# SELECT * FROM bus_station_sch.bilet WHERE "flight_ID" = 1 AND "ID_passenger" = 5 AND "place_number" = 8;
 ticket_ID | place_number | purchase_date | ticket_price | purchase_method | flight_ID | ID_passenger | stop_ID | ticket_status | tariff_id | has_hand_lugga
ge
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
(0 строк)
```

Ничего не вставилось, потому что дата не подходит.

Попробую вставить дату после рейса:

```
CALL bus_station_sch.pr_sale_ticket(8,'2026-04-11', 'K',1,5,3,1,false);
```

```

bus_station=# CALL bus_station_sch.pr_sale_ticket(8,'2026-04-11', 'K',1,5,3,1,false);
CALL
bus_station=# SELECT * FROM bus_station_sch.bilet WHERE "flight_ID" = 1 AND "ID_passenger" = 5 AND "place_number" = 8;
 ticket_ID | place_number | purchase_date | ticket_price | purchase_method | flight_ID | ID_passenger | stop_ID | ticket_status | tariff_id | has_hand_lugga
ge
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
(0 строк)

bus_station=#

```

Ничего не вставилось.

Попробую вставить корректную дату:

```

bus_station=# CALL bus_station_sch.pr_sale_ticket(8,'2026-03-31', 'K',1,5,3,1,false);
CALL
bus_station=# SELECT * FROM bus_station_sch.bilet WHERE "flight_ID" = 1 AND "ID_passenger" = 5 AND "place_number" = 8;
 ticket_ID | place_number | purchase_date | ticket_price | purchase_method | flight_ID | ID_passenger | stop_ID | ticket_status | tariff_id | has_hand_lugga
ge
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
85 | 8 | 2026-03-31 | 1000.00 | K | 1 | 5 | 3 | PAID | 1 | f
(1 строка)

bus_station=#

```

Всё вставилось.

Запрет продажи занятого места

На одном рейсе одно и то же место нельзя продать двум пассажирам.

```
CREATE OR REPLACE FUNCTION bus_station_sch.fn_bilet_seat_busy()
RETURNS trigger AS $$ DECLARE count_tickets integer; BEGIN IF
NEW."ticket_status" = 'PAID' AND NEW."place_number" IS NOT NULL THEN
SELECT count(*) INTO count_tickets FROM bus_station_sch.bilet WHERE
"flight_ID" = NEW."flight_ID" AND "place_number" = NEW."place_number"
AND "ticket_status" = 'PAID' AND "ticket_ID" <> NEW."ticket_ID"; IF
count_tickets > 0 THEN RETURN NULL; END IF; END IF; RETURN NEW;
END; $$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER tr_bilet_seat_busy BEFORE INSERT OR UPDATE ON
bus_station_sch.bilet FOR EACH ROW EXECUTE PROCEDURE
bus_station_sch.fn_bilet_seat_busy();
```

ПРОВЕРКА:

Попробуем купить место 8 на рейс 1 (в прошлом запросе я купила это место)

Вызовем процедуру:

```
CALL bus_station_sch.pr_sale_ticket(8,'2026-04-11', 'K',1,10,3,1,false);
```

```
bus_station=# CALL bus_station_sch.pr_sale_ticket(8,'2026-04-01', 'K',1,10,3,1,false);
CALL
bus_station=# SELECT * FROM bus_station_sch.bilet WHERE "flight_ID" = 1 AND "ID_passenger" = 10 AND "place_number" = 8;
 ticket_ID | place_number | purchase_date | ticket_price | purchase_method | flight_ID | ID_passenger | stop_ID | ticket_status | tariff_id | has_hand_lugga
ge
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
(0 строк)

bus_station=# |
```

Ничего не вставилось, попробуем с корректными данными – поменяю место:

```
bus_station=# CALL bus_station_sch.pr_sale_ticket(10,'2026-04-01', 'K',1,10,3,1,false);
CALL
bus_station=# SELECT * FROM bus_station_sch.bilet WHERE "flight_ID" = 1 AND "ID_passenger" = 10 AND "place_number" = 10;
 ticket_ID | place_number | purchase_date | ticket_price | purchase_method | flight_ID | ID_passenger | stop_ID | ticket_status | tariff_id | has_hand_lugga
ge
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
      86 |          10 | 2026-04-01 |      1000.00 | K |          1 |          10 |          3 | PAID |          1 | f
(1 строка)

bus_station=# |
```

Всё корректно вставилось, потому что место 10 не занято.

Ограничение билетов без места

По заданию на каждый рейс можно продать не больше 10 билетов без места.

```
CREATE OR REPLACE FUNCTION bus_station_sch.fn_bilet_no_seat_limit()
RETURNS trigger AS $$ DECLARE count_tickets integer; BEGIN IF
NEW."ticket_status" = 'PAID' AND NEW."place_number" IS NULL THEN
SELECT count(*) INTO count_tickets FROM bus_station_sch.bilet WHERE
"flight_ID" = NEW."flight_ID" AND "place_number" IS NULL AND
"ticket_status" = 'PAID' AND "ticket_ID" <> NEW."ticket_ID"; IF count_tickets
>= 10 THEN RETURN NULL; END IF; END IF; RETURN NEW; END; $$
LANGUAGE plpgsql;
```

```
CREATE TRIGGER tr_bilet_no_seat_limit BEFORE INSERT OR UPDATE ON
bus_station_sch.bilet FOR EACH ROW EXECUTE PROCEDURE
bus_station_sch.fn_bilet_no_seat_limit();
```

ПРОВЕРКА:

нужно взять один рейс и попробовать продать на него 11 билетов без места.

1-10 билеты без места должны добавиться.

11-й билет без места не должен добавиться.

На первом рейсе у меня 0 билетов без места:

```
no_seat_tickets
-----
0
(1 строка)
```

На нем буду тестировать.

Вставила 10 билетов где место NULL:

```
bus_station=# CALL bus_station_sch.pr_sale_ticket(NULL, '2026-04-01', 'K', 1, 14, 3, 1, false);
CALL
bus_station=# CALL bus_station_sch.pr_sale_ticket(NULL, '2026-04-01', 'K', 1, 15, 3, 1, false);
CALL
bus_station=# SELECT COUNT(*) AS no_seat_tickets FROM bus_station_sch.bilet WHERE "flight_ID" = 1 AND "place_number" IS NULL AND "ticket_status" = 'PAID';
no_seat_tickets
-----
10
(1 строка)
```

Пытаюсь вставить 11:

```
CALL bus_station_sch.pr_sale_ticket(NULL, '2026-04-01', 'K', 1, 16, 3, 1, false);
```

```
bus_station=# CALL bus_station_sch.pr_sale_ticket(NULL, '2026-04-01', 'K', 1, 16, 3, 1, false);
CALL
bus_station=# SELECT COUNT(*) AS no_seat_tickets FROM bus_station_sch.bilet WHERE "flight_ID" = 1 AND "place_number" IS NULL AND "ticket_status" = 'PAID';
no_seat_tickets
-----
10
(1 строка)
```

Триггер сработал и не дал купить 11-ый билет без места.

Запрет физического удаления оплаченного билета

если билет уже оплачен, его нельзя просто удалить из базы. Его можно только вернуть через процедуру возврата, то есть поменять статус на RETURNED. Это реально важно для автовокзала, потому что оплаченный билет это финансовая операция. Его нельзя стирать, иначе потеряется история продажи.

```
CREATE OR REPLACE FUNCTION bus_station_sch.fn_bilet_delete_paid()
RETURNS trigger AS $$ BEGIN IF OLD."ticket_status" = 'PAID' THEN
RETURN NULL; END IF; RETURN OLD; END; $$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER tr_bilet_delete_paid BEFORE DELETE ON
bus_station_sch.bilet FOR EACH ROW EXECUTE PROCEDURE
bus_station_sch.fn_bilet_delete_paid();
```

ПРОВЕРКА:

Пробую удалить оплаченный билет:

```
bus_station=# DELETE FROM bus_station_sch.bilet WHERE "ticket_ID" = 10;
DELETE 0
bus_station=# SELECT "ticket_ID","ticket_status" FROM bus_station_sch.bilet WHERE "ticket_ID" = 10;
 ticket_ID | ticket_status
-----+-----
          10 | PAID
(1 строка)

bus_station=# |
```

Удаление не произошло, триггер сработал.

Защита возвращенного билета

если билет уже возвращен, его нельзя снова сделать активным или поменять данные билета.

```
CREATE OR REPLACE FUNCTION bus_station_sch.fn_bilet_returned_protect()  
RETURNS trigger AS $$ BEGIN IF OLD."ticket_status" = 'RETURNED' THEN  
IF NEW."ticket_status" IS DISTINCT FROM OLD."ticket_status" THEN  
RETURN NULL; END IF; IF NEW."flight_ID" IS DISTINCT FROM  
OLD."flight_ID" THEN RETURN NULL; END IF; IF NEW."ID_passenger" IS  
DISTINCT FROM OLD."ID_passenger" THEN RETURN NULL; END IF; IF  
NEW."place_number" IS DISTINCT FROM OLD."place_number" THEN  
RETURN NULL; END IF; IF NEW."stop_ID" IS DISTINCT FROM  
OLD."stop_ID" THEN RETURN NULL; END IF; IF NEW."ticket_price" IS  
DISTINCT FROM OLD."ticket_price" THEN RETURN NULL; END IF; IF  
NEW."tariff_id" IS DISTINCT FROM OLD."tariff_id" THEN RETURN NULL;  
END IF; END IF; RETURN NEW; END; $$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER tr_bilet_returned_protect BEFORE UPDATE ON  
bus_station_sch.bilet FOR EACH ROW EXECUTE PROCEDURE  
bus_station_sch.fn_bilet_returned_protect();
```

ПРОВЕРКА:

Возьму существующий билет, сделаю его возвращенным, то есть ticket_status = RETURNED, попробую снова сделать его PAID, изменить данные билета.

У меня есть возвращённый билет:

ticket_ID = 84

ticket_status = RETURNED

ВОТ С ТАКИМИ ДАННЫМИ:

```
bus_station=# SELECT * FROM bus_station_sch.bilet WHERE "ticket_ID" = 84;  
ticket_ID | place_number | purchase_date | ticket_price | purchase_method | flight_ID | ID_passenger | stop_ID | ticket_status | tariff_id | has_hand_luggage  
-----  
84 | 6 | 2026-05-08 | 1000.00 | 0 | 1 | 1 | 1 | RETURNED | 1 | f  
(1 строка)
```

Проверяю, что билет снова нельзя сделать активным:

```
UPDATE bus_station_sch.bilet SET "ticket_status" = 'PAID' WHERE "ticket_ID"  
= 84;
```

```

bus_station=# UPDATE bus_station_sch.bilet SET "ticket_status" = 'PAID' WHERE "ticket_ID" = 84;
UPDATE 0
bus_station=# SELECT * FROM bus_station_sch.bilet WHERE "ticket_ID" = 84;
 ticket_ID | place_number | purchase_date | ticket_price | purchase_method | flight_ID | ID_passenger | stop_ID | ticket_status | tariff_id | has_hand_luggage
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
          84 |           6 | 2026-05-08   |      1000.00 | 0               |          1 |           1 |          1 | RETURNED    |          1 | f
(1 строка)

bus_station=#
bus_station=# SELECT "ticket_ID", "ticket_status" FROM bus_station_sch.bilet WHERE "ticket_ID" = 84;
 ticket_ID | ticket_status
-----+-----
          84 | RETURNED
(1 строка)

```

Проверяю, что нельзя изменить место возвращенного билета.

```
UPDATE bus_station_sch.bilet SET "place_number" = 7 WHERE "ticket_ID" = 84;
```

```

bus_station=# UPDATE bus_station_sch.bilet SET "place_number" = 7 WHERE "ticket_ID" = 84;
UPDATE 0
bus_station=# SELECT "ticket_ID", "place_number", "ticket_status" FROM bus_station_sch.bilet WHERE "ticket_ID" = 84;
 ticket_ID | place_number | ticket_status
-----+-----+-----
          84 |           6 | RETURNED
(1 строка)

```

Проверяю, что нельзя изменить пассажира:

```
UPDATE bus_station_sch.bilet SET "ID_passenger" = 2 WHERE "ticket_ID" = 84;
```

```

bus_station=# UPDATE bus_station_sch.bilet SET "ID_passenger" = 2 WHERE "ticket_ID" = 84;
UPDATE 0
bus_station=# SELECT "ticket_ID", "ID_passenger", "ticket_status" FROM bus_station_sch.bilet WHERE "ticket_ID" = 84;
 ticket_ID | ID_passenger | ticket_status
-----+-----+-----
          84 |           1 | RETURNED
(1 строка)

bus_station=#

```

Триггер не даёт делать обновление.

Логирование операций с билетами

записывает в logs, что билет был добавлен, изменен или удален.

```
CREATE OR REPLACE FUNCTION bus_station_sch.fn_bilet_log() RETURNS
trigger AS $$ DECLARE mstr varchar(100); astr varchar(100); retstr varchar(254);
BEGIN IF TG_OP = 'INSERT' THEN astr := NEW."ticket_ID"; mstr := 'Add
ticket '; retstr := mstr || astr; INSERT INTO bus_station_sch.logs(text,added)
VALUES(retstr,now()); RETURN NEW; ELSIF TG_OP = 'UPDATE' THEN astr
:= NEW."ticket_ID"; mstr := 'Update ticket '; retstr := mstr || astr; INSERT INTO
bus_station_sch.logs(text,added) VALUES(retstr,now()); RETURN NEW; ELSIF
TG_OP = 'DELETE' THEN astr := OLD."ticket_ID"; mstr := 'Delete ticket '; retstr
:= mstr || astr; INSERT INTO bus_station_sch.logs(text,added)
VALUES(retstr,now()); RETURN OLD; END IF; END; $$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER tr_bilet_log AFTER INSERT OR UPDATE OR DELETE
ON bus_station_sch.bilet FOR EACH ROW EXECUTE PROCEDURE
bus_station_sch.fn_bilet_log();
```

ПРОВЕРКА:

Добавлю новый билет:

```
bus_station=# CALL bus_station_sch.pr_sale_ticket(20, '2026-04-01', 'K', 2, 16, 6, 1, false);
CALL
bus_station=# SELECT * FROM bus_station_sch.bilet WHERE "flight_ID" = 2 AND "ID_passenger" = 16 AND "place_number" = 20;
 ticket_ID | place_number | purchase_date | ticket_price | purchase_method | flight_ID | ID_passenger | stop_ID | ticket_status | tariff_id | has_hand_lug
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
          97 |           20 | 2026-04-01   |      1000.00 | K               |          2 |          16 |         6 | PAID         |           | f
(1 строка)
```

```
SELECT * FROM bus_station_sch.logs ORDER BY added DESC LIMIT 5;
```

```
bus_station=# SELECT * FROM bus_station_sch.logs ORDER BY added DESC LIMIT 5;
   text   | added
-----+-----
Add ticket 97 | 2026-05-08 11:13:27.75271
```

Появилось в таблице.

Обновим билет, поменяем способ покупки:

```
UPDATE bus_station_sch.bilet SET "purchase_method" = 'O' WHERE
"ticket_ID" = 97;
```

Опять проверяю логи:

```
bus_station=# SELECT * FROM bus_station_sch.logs ORDER BY added DESC LIMIT 5;
   text   | added
-----+-----
Update ticket 97 | 2026-05-08 11:17:18.676476
Add ticket 97 | 2026-05-08 11:13:27.75271
```

И теперь удалю билет:

```

bus_station=# DELETE FROM bus_station_sch.bilet WHERE "ticket_ID" = 97;
DELETE 0
bus_station=# CALL bus_station_sch.pr_return_ticket(97);
CALL
bus_station=# SELECT "ticket_ID", "ticket_status" FROM bus_station_sch.bilet WHERE "ticket_ID" = 97;
 ticket_ID | ticket_status
-----+-----
          97 | RETURNED
(1 строка)

bus_station=# SELECT * FROM bus_station_sch.logs ORDER BY added DESC LIMIT 5;
      text      |      added
-----+-----
Update ticket 97 | 2026-05-08 11:19:34.527117
Update ticket 97 | 2026-05-08 11:17:18.676476
Add ticket 97    | 2026-05-08 11:13:27.75271

```

Сначала пришлось поменять его статус, затем удалить

```

bus_station=# DELETE FROM bus_station_sch.bilet WHERE "ticket_ID" = 97 AND "ticket_status" = 'RETURNED';
DELETE 1
bus_station=# SELECT * FROM bus_station_sch.logs ORDER BY added DESC LIMIT 5;
      text      |      added
-----+-----
Delete ticket 97 | 2026-05-08 11:21:39.603221
Update ticket 97 | 2026-05-08 11:19:34.527117
Update ticket 97 | 2026-05-08 11:17:18.676476
Add ticket 97    | 2026-05-08 11:13:27.75271

```

Автобус не может быть на двух рейсах одновременно

один автобус нельзя назначить на два рейса в одну дату и одно время.

```
CREATE OR REPLACE FUNCTION bus_station_sch.fn_reis_bus_busy()
RETURNS trigger AS $$ DECLARE count_reis integer; BEGIN SELECT
count(*) INTO count_reis FROM bus_station_sch.reis WHERE "bus_ID" =
NEW."bus_ID" AND "departure_date" = NEW."departure_date" AND
"departure_time" = NEW."departure_time" AND "flight_ID" <>
NEW."flight_ID"; IF count_reis > 0 THEN RETURN NULL; END IF; RETURN
NEW; END; $$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER tr_reis_bus_busy BEFORE INSERT OR UPDATE ON
bus_station_sch.reis FOR EACH ROW EXECUTE PROCEDURE
bus_station_sch.fn_reis_bus_busy();
```

ПРОВЕРКА:

Возьму существующий рейс:

flight_ID = 1

departure_date = 2026-04-10

departure_time = 08:30:00

bus_ID = BUS001

destination_ID = 2

Пробую создать новый рейс на ту же дату и то же время с тем же автобусом BUS001.

INSERT INTO

```
bus_station_sch.reis("flight_ID","departure_date","departure_time","flight_type","
bus_ID","destination_ID")VALUES (100, '2026-04-10','08:30:00','R','BUS001',2);
```

```
bus_station=# INSERT INTO bus_station_sch.reis("flight_ID","departure_date","departure_time","flight_type","bus_ID","destination_ID")VALUES (100, '2026-04-
10','08:30:00','R','BUS001',2);
INSERT 0 0
bus_station=#
```

Не вставилось.

```
bus_station=# SELECT * FROM bus_station_sch.reis WHERE "flight_ID" = 100;
 flight_ID | departure_date | departure_time | flight_type | bus_ID | destination_ID
-----+-----+-----+-----+-----+-----
(0 строк)
```

Проверка корректного варианта

Например, автобус BUS002 существует. У него есть рейс 2, но в другое время: 2026-04-10 10:45:00

А я поставлю 2026-04-10 08:30:00

INSERT INTO

bus_station_sch.reis("flight_ID","departure_date","departure_time","flight_type","
bus_ID","destination_ID")VALUES (101,'2026-04-10','08:30:00','R','BUS002', 3);

```
bus_station=# SELECT * FROM bus_station_sch.reis WHERE "flight_ID" = 101;
 flight_ID | departure_date | departure_time | flight_type | bus_ID | destination_ID
-----+-----+-----+-----+-----+-----
        101 | 2026-04-10    | 08:30:00      | R          | BUS002 |          3
(1 строка)
```

Он спокойно изменился, потому что стоит другой автобус.

Контроль изменения рейса

можно менять автобус; можно менять дату и время; нельзя удалять рейс, если по нему есть билеты; при замене автобуса новый автобус должен вместить уже проданные места; если рейс с билетами изменили, это записывается в logs.

```
CREATE OR REPLACE FUNCTION bus_station_sch.fn_reis_update_control()
RETURNS trigger AS $$ DECLARE count_tickets integer; max_place integer;
new_bus_places integer; last_purchase_date date; BEGIN SELECT count(*) INTO
count_tickets FROM bus_station_sch.bilet WHERE "flight_ID" =
OLD."flight_ID"; IF TG_OP = 'DELETE' THEN IF count_tickets > 0 THEN
INSERT INTO bus_station_sch.logs(text,added) VALUES('Try delete flight with
tickets ' || OLD."flight_ID",now()); RETURN NULL; END IF; RETURN OLD;
END IF; IF count_tickets > 0 THEN SELECT max("place_number") INTO
max_place FROM bus_station_sch.bilet WHERE "flight_ID" = OLD."flight_ID"
AND "ticket_status" = 'PAID'; SELECT max("purchase_date") INTO
last_purchase_date FROM bus_station_sch.bilet WHERE "flight_ID" =
OLD."flight_ID" AND "ticket_status" = 'PAID'; IF NEW."departure_date" <
last_purchase_date THEN RETURN NULL; END IF; IF NEW."bus_ID" IS
DISTINCT FROM OLD."bus_ID" THEN SELECT m."place_number" INTO
new_bus_places FROM bus_station_sch.avtobus a JOIN
bus_station_sch.model_avtobusa m ON a."model_ID" = m."model_ID" WHERE
a."bus_ID" = NEW."bus_ID"; IF new_bus_places IS NULL THEN RETURN
NULL; END IF; IF max_place IS NOT NULL AND new_bus_places < max_place
THEN RETURN NULL; END IF; END IF; INSERT INTO
bus_station_sch.logs(text,added) VALUES('Update flight with tickets ' ||
OLD."flight_ID",now()); END IF; RETURN NEW; END; $$ LANGUAGE
plpgsql;
```

```
CREATE TRIGGER tr_reis_update_control BEFORE UPDATE OR DELETE
ON bus_station_sch.reis FOR EACH ROW EXECUTE PROCEDURE
bus_station_sch.fn_reis_update_control();
```

ПРОВЕРКА:

Пробую удалить рейс с билетами:

```
DELETE FROM bus_station_sch.reis WHERE "flight_ID" = 1;
```

```

bus_station=# DELETE FROM bus_station_sch.reis WHERE "flight_ID" = 1;
DELETE 0
bus_station=# SELECT * FROM bus_station_sch.reis WHERE "flight_ID" = 1;
 flight_ID | departure_date | departure_time | flight_type | bus_ID | destination_ID
-----+-----+-----+-----+-----+-----
          1 | 2026-04-10    | 08:30:00      | R          | BUS001 |              2
(1 строка)

bus_station=# SELECT * FROM bus_station_sch.logs ORDER BY added DESC;
      text      | added
-----+-----
Try delete flight with tickets 1 | 2026-05-08 11:53:18.311692

```

Меняю время: UPDATE bus_station_sch.reis SET "departure_time" = TIME '12:00' WHERE "flight_ID" = 1;

```

bus_station=# UPDATE bus_station_sch.reis SET "departure_time" = TIME '12:00' WHERE "flight_ID" = 1;
UPDATE 1
bus_station=# SELECT "flight_ID","departure_date","departure_time","bus_ID" FROM bus_station_sch.reis WHERE "flight_ID" = 1;
 flight_ID | departure_date | departure_time | bus_ID
-----+-----+-----+-----
          1 | 2026-04-10    | 12:00:00      | BUS001
(1 строка)

```

Меняю автобус:

UPDATE bus_station_sch.reis SET "bus_ID" = 'BUS003' WHERE "flight_ID" = 1;

```

bus_station=# UPDATE bus_station_sch.reis SET "bus_ID" = 'BUS003' WHERE "flight_ID" = 1;
UPDATE 1
bus_station=# SELECT "flight_ID","bus_ID" FROM bus_station_sch.reis WHERE "flight_ID" = 1;
 flight_ID | bus_ID
-----+-----
          1 | BUS003
(1 строка)

```

Проверка медосмотра экипажа

водитель должен иметь корректную дату и статус медосмотра. Если водитель не допущен, должна быть указана причина.

```
CREATE OR REPLACE FUNCTION bus_station_sch.fn_ekipazh_med()
RETURNS trigger AS $$ DECLARE flight_date date; BEGIN SELECT
"departure_date" INTO flight_date FROM bus_station_sch.reis WHERE
"flight_ID" = NEW."flight_ID"; IF flight_date IS NULL THEN RETURN NULL;
END IF; IF NEW."date_medical_examination" IS NULL THEN RETURN NULL;
END IF; IF NEW."date_medical_examination" > flight_date THEN RETURN
NULL; END IF; IF NEW."medical_check_up_status" NOT IN ('P','N') THEN
RETURN NULL; END IF; IF NEW."medical_check_up_status" = 'N' AND
NEW."reason_non_admission" IS NULL THEN RETURN NULL; END IF; IF
NEW."medical_check_up_status" = 'N' AND NEW."reason_non_admission" = "
THEN RETURN NULL; END IF; IF NEW."medical_check_up_status" = 'P'
THEN NEW."reason_non_admission" := NULL; END IF; RETURN NEW; END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER tr_ekipazh_med BEFORE INSERT OR UPDATE ON
bus_station_sch.ekipazh FOR EACH ROW EXECUTE PROCEDURE
bus_station_sch.fn_ekipazh_med();
```

ПРОВЕРКА:

нельзя поставить дату медосмотра позже даты рейса

```
bus_station=# FROM bus_station_sch.ekipazh
bus_station=# WHERE "ID_crew" = 1;
 ID_crew | flight_ID | driver_ID | driver_role | date_medical_examination | medical_check_up_status | reason_non_admission
-----+-----+-----+-----+-----+-----+-----
      1 |          |      2 | второй водитель | 2026-04-10                | P                        |
(1 строка)
```

```
bus_station=# SELECT *
bus_station=# FROM bus_station_sch.reis
bus_station=# WHERE "flight_ID" = 1;
 flight_ID | departure_date | departure_time | flight_type | bus_ID | destination_ID
-----+-----+-----+-----+-----+-----
      1 | 2026-04-10    | 12:00:00      | R          | BUS003 |          2
(1 строка)
```

Рейс отправляется 2026-04-10.

Пробую поставить медосмотр на 2026-04-11

```
UPDATE bus_station_sch.ekipazh SET "date_medical_examination" = '2026-04-11' WHERE "ID_crew" = 1;
```

Проверяем:

```
SELECT
"ID_crew","flight_ID","date_medical_examination","medical_check_up_status","reason_non_admission" FROM bus_station_sch.ekipazh WHERE "ID_crew" = 1;
```

```

bus_station=# SELECT "ID_crew","flight_ID","date_medical_examination","medical_check_up_status","reason_non_admission" FROM bus_station_sch.ekipazh WHERE "ID_crew" = 1;
ID_crew | flight_ID | date_medical_examination | medical_check_up_status | reason_non_admission
-----+-----+-----+-----+-----
      1 |      1 | 2026-04-10              | P                        |
(1 строка)

```

Дата не поменялась.

нельзя оставить дату медосмотра пустой

```

UPDATE bus_station_sch.ekipazh SET "date_medical_examination" = NULL
WHERE "ID_crew" = 1;

```

```

bus_station=# UPDATE bus_station_sch.ekipazh SET "date_medical_examination" = NULL WHERE "ID_crew" = 1;
UPDATE 0
bus_station=# SELECT "ID_crew","flight_ID","date_medical_examination","medical_check_up_status","reason_non_admission" FROM bus_station_sch.ekipazh WHERE "ID_crew" = 1;
ID_crew | flight_ID | date_medical_examination | medical_check_up_status | reason_non_admission
-----+-----+-----+-----+-----
      1 |      1 | 2026-04-10              | P                        |
(1 строка)

```

Ничего не поменялось.

если водитель не допущен, должна быть причина:

пробовала поставить статус N, но причину не указывать:

```

UPDATE bus_station_sch.ekipazh SET "medical_check_up_status" = 'N',
"reason_non_admission" = NULL WHERE "ID_crew" = 1;

```

```

bus_station=# UPDATE bus_station_sch.ekipazh SET "medical_check_up_status" = 'N', "reason_non_admission" = NULL WHERE "ID_crew" = 1;
UPDATE 0
bus_station=# SELECT "ID_crew"

```

```

ID_crew | medical_check_up_status | reason_non_admission
-----+-----+-----
      1 | N                        |
(1 строка)

```

если причина пустая строка, тоже нельзя

```

UPDATE bus_station_sch.ekipazh SET "medical_check_up_status" = 'N',
"reason_non_admission" = '' WHERE "ID_crew" = 1;

```

```

bus_station=# UPDATE bus_station_sch.ekipazh SET "medical_check_up_status" = 'N', "reason_non_admission" = '' WHERE "ID_crew" = 1;
UPDATE 0
bus_station=#

```

если водитель не допущен и причина есть, запись должна измениться

```

UPDATE bus_station_sch.ekipazh SET "medical_check_up_status" = 'N',
"reason_non_admission" = 'Высокое давление' WHERE "ID_crew" = 1;

```

```

bus_station=# UPDATE bus_station_sch.ekipazh SET "medical_check_up_status" = 'N', "reason_non_admission" = 'Высокое давление' WHERE "ID_crew" = 1;
UPDATE 1
bus_station=#

```

```

ID_crew | medical_check_up_status | reason_non_admission
-----+-----+-----
      1 | N                        | Высокое давление
(1 строка)

```

Изменилось.

если водитель допущен, причина должна автоматически стать NULL
верну водителю обратно статус Р

UPDATE bus_station_sch.ekipazh SET "medical_check_up_status" = 'P',
"reason_non_admission" = 'Тестовая причина' WHERE "ID_crew" = 1;

```
bus_station=# UPDATE bus_station_sch.ekipazh SET "medical_check_up_status" = 'P', "reason_non_admission" = 'Тестовая причина' WHERE "ID_crew" = 1;
UPDATE 1
```

ID_crew	flight_ID	date_medical_examination	medical_check_up_status	reason_non_admission
1	1	2026-04-10	P	

(1 строка)

Контроль экипажа

На рейсе максимум 2 допущенных водителя.

У допущенных водителей не должны повторяться роли. Один и тот же водитель не должен быть назначен на один рейс дважды. Один водитель не должен быть назначен на два рейса в одну дату и время.

```
CREATE OR REPLACE FUNCTION bus_station_sch.fn_ekipazh_control()
RETURNS trigger AS $$ DECLARE count_drivers integer; count_roles integer;
count_same_driver integer; count_driver_time integer; BEGIN IF
NEW."medical_check_up_status" = 'P' THEN IF NEW."driver_role" NOT IN
('первый водитель','второй водитель') THEN RETURN NULL; END IF;
SELECT count(*) INTO count_drivers FROM bus_station_sch.ekipazh WHERE
"flight_ID" = NEW."flight_ID" AND "medical_check_up_status" = 'P' AND
"ID_crew" <> NEW."ID_crew"; IF count_drivers >= 2 THEN RETURN NULL;
END IF; SELECT count(*) INTO count_roles FROM bus_station_sch.ekipazh
WHERE "flight_ID" = NEW."flight_ID" AND "driver_role" = NEW."driver_role"
AND "medical_check_up_status" = 'P' AND "ID_crew" <> NEW."ID_crew"; IF
count_roles > 0 THEN RETURN NULL; END IF; SELECT count(*) INTO
count_same_driver FROM bus_station_sch.ekipazh WHERE "flight_ID" =
NEW."flight_ID" AND "driver_ID" = NEW."driver_ID" AND
"medical_check_up_status" = 'P' AND "ID_crew" <> NEW."ID_crew"; IF
count_same_driver > 0 THEN RETURN NULL; END IF; SELECT count(*)
INTO count_driver_time FROM bus_station_sch.ekipazh e JOIN
bus_station_sch.reis r1 ON e."flight_ID" = r1."flight_ID" JOIN
bus_station_sch.reis r2 ON r2."flight_ID" = NEW."flight_ID" WHERE
e."driver_ID" = NEW."driver_ID" AND e."medical_check_up_status" = 'P' AND
e."ID_crew" <> NEW."ID_crew" AND r1."departure_date" = r2."departure_date"
AND r1."departure_time" = r2."departure_time"; IF count_driver_time > 0 THEN
RETURN NULL; END IF; END IF; RETURN NEW; END; $$ LANGUAGE
plpgsql;
```

```
CREATE TRIGGER tr_ekipazh_control BEFORE INSERT OR UPDATE ON
bus_station_sch.ekipazh FOR EACH ROW EXECUTE PROCEDURE
bus_station_sch.fn_ekipazh_control();
```

ПРОВЕРКА:

У меня есть:

flight_ID = 20

departure_date = 2026-04-11

departure_time = 16:50:00

В экипаже рейса 20 сейчас:

ID_crew = 39, driver_ID = 2, второй водитель, статус N

ID_crew = 40, driver_ID = 1, первый водитель, статус P

То есть на рейсе 20 сейчас только один допущенный водитель. Это удобно для проверки.

```
bus_station=# ORDER BY "ID_crew";
```

ID_crew	flight_ID	driver_ID	driver_role	date_medical_examination	medical_check_up_status	reason_non_admission
39	20	2	второй водитель	2026-04-11	N	Высокое давление
40	20	1	первый водитель	2026-04-11	P	

(2 строки)

flight_ID	departure_date	departure_time	flight_type	bus_ID	destination_ID
20	2026-04-11	16:50:00	R	BUS005	3

(1 строка)

Проверка, что нельзя повторять роль допущенного водителя

Попробую добавить еще одного допущенного водителя с такой же ролью как у первого водителя:

INSERT INTO

bus_station_sch.ekipazh("ID_crew","flight_ID","driver_ID","driver_role","date_medical_examination","medical_check_up_status","reason_non_admission")
VALUES (100,20,2,'первый водитель','2026-04-11','P',NULL);

```
bus_station=# INSERT INTO bus_station_sch.ekipazh("ID_crew","flight_ID","driver_ID","driver_role","date_medical_examination","medical_check_up_status","reason_non_admission") VALUES (100,20,2,'первый водитель','2026-04-11','P',NULL);
INSERT 0 0
bus_station=# SELECT * FROM bus_station_sch.ekipazh WHERE "ID_crew" = 100;
```

ID_crew	flight_ID	driver_ID	driver_role	date_medical_examination	medical_check_up_status	reason_non_admission
---------	-----------	-----------	-------------	--------------------------	-------------------------	----------------------

(0 строк)

один водитель не может быть назначен на один рейс дважды

На рейсе 20 уже есть допущенный водитель: driver_ID = 1

Пробую назначить этого же водителя на этот же рейс еще раз, но уже как второго водителя:

INSERT INTO

bus_station_sch.ekipazh("ID_crew","flight_ID","driver_ID","driver_role","date_medical_examination","medical_check_up_status","reason_non_admission")VALUES (101,20,1,'второй водитель','2026-04-11','P',NULL);

```
bus_station=# INSERT INTO bus_station_sch.ekipazh("ID_crew","flight_ID","driver_ID","driver_role","date_medical_examination","medical_check_up_status","reason_non_admission")VALUES (101,20,1,'второй водитель','2026-04-11','P',NULL);
INSERT 0 0
```

То есть триггер запретил назначить одного и того же водителя на один рейс дважды.

корректное добавление второго допущенного водителя

у меня есть запись:

ID_crew = 39

flight_ID = 20

driver_ID = 2

driver_role = второй водитель

medical_check_up_status = N

сделаю этого водителя допущенным

```
UPDATE bus_station_sch.ekipazh SET "medical_check_up_status" = 'P',  
"reason_non_admission" = NULL WHERE "ID_crew" = 39;
```

```
bus_station=# UPDATE bus_station_sch.ekipazh SET "medical_check_up_status" = 'P', "reason_non_admission" = NULL WHERE "ID_crew" = 39;  
UPDATE 1
```

ID_crew	flight_ID	driver_ID	driver_role	date_medical_examination	medical_check_up_status	reason_non_admission
39	20	2	второй водитель	2026-04-11	P	
40	20	1	первый водитель	2026-04-11	P	

(2 строки)

на рейсе не может быть больше двух допущенных водителей

INSERT INTO

```
bus_station_sch.ekipazh("ID_crew","flight_ID","driver_ID","driver_role","date_m  
edical_examination","medical_check_up_status","reason_non_admission")  
VALUES (102,20,3,'первый водитель','2026-04-11','P',NULL);
```

```
bus_station=# INSERT INTO bus_station_sch.ekipazh("ID_crew","flight_ID","driver_ID","driver_role","date_medical_examination","medical_check_up_status","rea  
son_non_admission") VALUES (102,20,3,'первый водитель','2026-04-11','P',NULL);  
INSERT 0 0
```

ID_crew	flight_ID	driver_ID	driver_role	date_medical_examination	medical_check_up_status	reason_non_admission

(0 строк)

один водитель не может быть на двух рейсах в одну дату и время

Для этой проверки нужно создать временный рейс с такой же датой и временем, как у рейса 20. Рейс 20:2026-04-11 16:50:00

INSERT INTO

```
bus_station_sch.reis("flight_ID","departure_date","departure_time","flight_type",  
bus_ID,"destination_ID")VALUES (200,'2026-04-11','16:50:00','R','BUS002',3);
```

flight_ID	departure_date	departure_time	flight_type	bus_ID	destination_ID
200	2026-04-11	16:50:00	R	BUS002	3

(1 строка)

Теперь пробуем назначить на новый рейс водителя 1.

Но водитель 1 уже допущен на рейс 20, а рейс 20 идет в ту же дату и время:

INSERT INTO

```
bus_station_sch.ekipazh("ID_crew","flight_ID","driver_ID","driver_role","date_m
```

```
edical_examination","medical_check_up_status","reason_non_admission")  
VALUES (103,200,1,'первый водитель','2026-04-11','P',NULL);
```

```
bus_station=# INSERT INTO bus_station_sch.ekipazh("ID_crew","flight_ID","driver_ID","driver_role","date_medical_examination","medical_check_up_status","reason_non_admission") VALUES (103,200,1,'первый водитель','2026-04-11','P',NULL);  
INSERT 0 0  
bus_station=# |
```

```
bus_station=# WHERE ID_crew = 103;  
ID_crew | flight_ID | driver_ID | driver_role | date_medical_examination | medical_check_up_status | reason_non_admission  
-----+-----+-----+-----+-----+-----+-----  
(0 строк)
```

триггер запретил назначить одного водителя на два рейса, которые отправляются в одну дату и одно время.